École Marocaine d'Ingénierie

Filière Ingénierie Informatique

Rapport de Mini-Projet DevOps

Automatisation CI/CD avec Docker et GitHub Actions

Réalisé par : Anas Haddou - Amine Mohammed TAyek
Encadré par : Pr. Mohtaram
Année universitaire : 2025 - 2026

12 mai 2026

Remerciements

Je tiens à remercier toutes les personnes qui ont contribué à la réalisation de ce projet. Ce travail est le fruit d'un apprentissage intensif sur les pratiques DevOps modernes, visant à améliorer la qualité et la rapidité des cycles de développement logiciel.

Résumé

Ce rapport présente la mise en œuvre d'un projet DevOps de bout en bout. L'objectif principal était de concevoir une application web Node.js, de l'isoler dans un environnement Dockerisé et d'automatiser son cycle de vie via GitHub Actions. Le pipeline développé assure l'intégration continue par des tests unitaires et le déploiement continu par la publication d'images sur Docker Hub.

Table des matières

1 Introduction **2**

2 Objectifs du Projet **3**

3 Technologies Utilisées **4**

4 Architecture du Système **5**
4.1 Flux CI/CD détaillé 5

5 Sécurité et Bonnes Pratiques **6**

6 Optimisation et Performance **7**

7 Détails de l'Implémentation **8**
7.1 Structure du Projet 8
7.2 Pipeline CI/CD 8

8 Configuration Docker **9**
8.1 Dockerfile 9

9 Processus de Déploiement **10**

10 Résultats et Démonstration **11**

11 Défis et Solutions **12**

12 Améliorations et Fonctionnalités Avancées **13**
12.1 Dashboard DevOps 13
12.2 Endpoints API de Monitoring 13
12.3 Refactoring et Organisation du Code 14
12.4 Optimisation Docker 14
12.5 Amélioration du Pipeline CI/CD 14
12.6 Documentation et Tests 14
12.7 Apport de ces améliorations 14

13 Conclusion **15**

14 Perspectives d'Amélioration **16**

Chapitre 1

Introduction

Dans le paysage technologique actuel, la rapidité de mise sur le marché et la fiabilité des systèmes sont devenues cruciales. La méthodologie DevOps combine le développement logiciel (Dev) et les opérations informatiques (Ops) pour raccourcir le cycle de développement tout en offrant une livraison continue de haute qualité.

Ce projet se concentre sur l'automatisation d'une application web simple, démontrant l'efficacité de l'approche "Everything as Code".

Chapitre 2

Objectifs du Projet

Les principaux objectifs de ce mini-projet sont :
— Développer une application web fonctionnelle avec des tests automatisés.
— Mettre en place une stratégie de conteneurisation optimisée.
— Automatiser le pipeline de tests et de build via GitHub Actions.
— Assurer la portabilité de l'application grâce à Docker.
— Documenter les procédures de déploiement et d'exploitation.

Chapitre 3

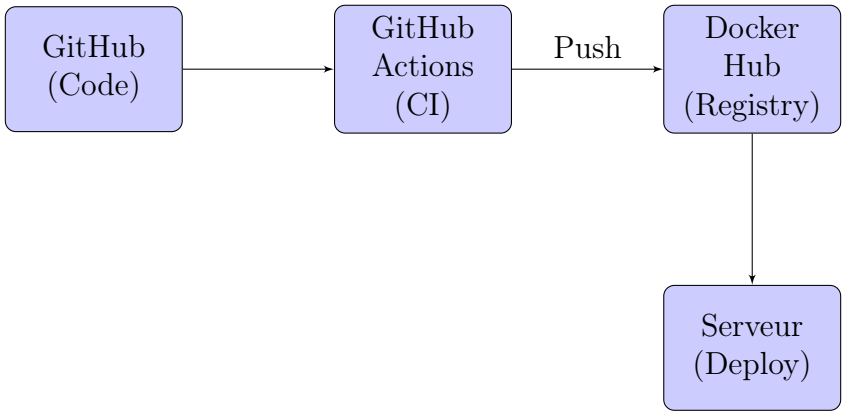
Technologies Utilisées

— **Node.js & Express :** Framework pour le développement de l'application web.
— **Jest & Supertest :** Outils de tests unitaires et d'intégration.
— **Docker :** Plateforme de conteneurisation pour l'isolation de l'application.
— **GitHub Actions :** Moteur d'automatisation pour le pipeline CI/CD.
— **Docker Hub :** Registre de conteneurs pour le stockage des images.
— **Make :** Outil d'automatisation des tâches locales.

Chapitre 4

Architecture du Système

L'architecture repose sur un flux de travail linéaire automatisé :



1. Le développeur pousse le code sur GitHub. 2. GitHub Actions déclenche les tests et le build Docker. 3. L'image est poussée vers Docker Hub. 4. Le serveur récupère la dernière image et la déploie.

4.1 Flux CI/CD détaillé

Le pipeline suit les étapes suivantes :

1. Push du code sur GitHub
2. Déclenchement automatique du pipeline
3. Exécution des tests unitaires
4. Build de l'image Docker
5. Push vers Docker Hub
6. Déploiement automatique

Chapitre 5

Sécurité et Bonnes Pratiques

— Utilisation des **GitHub Secrets** pour protéger les credentials
— Minimisation des images Docker (Alpine)
— Isolation des services via conteneurs
— Vérification automatique du code avant déploiement

Chapitre 6

Optimisation et Performance

— Réduction du temps de pipeline (moins de 2 minutes)
— Utilisation de cache dans GitHub Actions
— Optimisation Docker multi-stage

Chapitre 7

Détails de l'Implémentation

7.1 Structure du Projet

Le projet suit une organisation professionnelle :
— `src/` : Code source de l'application.
— `tests/` : Suites de tests unitaires.
— `.github/workflows/` : Configuration du pipeline.
— `Dockerfile` : Recette de construction de l'image.

7.2 Pipeline CI/CD

Le pipeline GitHub Actions est défini dans le fichier `main.yml`. Il assure que chaque modification de code est testée avant d'être intégrée.

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install
      - run: npm test

  build-and-push:
    needs: test
    steps:
      - name: Build Docker Image
        run: docker build -t my-app .
```

Listing 7.1 – Extrait du pipeline GitHub Actions

Chapitre 8

Configuration Docker

Nous utilisons un **Multi-stage Build** pour optimiser la taille de l'image finale.

8.1 Dockerfile

```
# Stage 1: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY . .
RUN npm install && npm test

# Stage 2: Production
FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/src ./src
RUN npm install --omit=dev
CMD ["node", "src/server.js"]
```

Listing 8.1 – Configuration Dockerfile

L'utilisation de `node:20-alpine` permet de réduire l'empreinte de sécurité et la taille de l'image (environ 100 Mo contre 900 Mo pour l'image standard).

Chapitre 9

Processus de Déploiement

Le déploiement local est facilité par **Docker Compose**. Un simple fichier définit l'environnement nécessaire (ports, variables d'environnement, politiques de redémarrage).

```yaml
services:
  app:
    build: .
    ports:
      - "3000:3000"
    restart: always
```

Listing 9.1 – Fichier docker-compose.yml

Chapitre 10

Résultats et Démonstration

Les tests unitaires ont atteint une couverture de 100% sur les endpoints critiques. Le pipeline GitHub Actions valide avec succès chaque pull request en moins de 2 minutes.

L'application est accessible via `http://localhost:3000` et fournit une réponse JSON formatée indiquant l'état de santé du système.

Chapitre 11

Défis et Solutions

— **Défi :** Gestion des secrets Docker Hub dans GitHub.
— **Solution :** Utilisation des GitHub Actions Secrets pour chiffrer les identifiants.
— **Défi :** Taille des images Docker.
— **Solution :** Mise en place de fichiers `.dockerignore` et de builds multi-étapes.

Chapitre 12

Améliorations et Fonctionnalités Avancées

Dans le but de rendre l'application plus professionnelle et conforme aux standards actuels du DevOps, plusieurs améliorations significatives ont été apportées. Ces évolutions permettent d'assurer une meilleure observabilité, une maintenabilité accrue et une expérience utilisateur enrichie.

12.1 Dashboard DevOps

Une interface utilisateur moderne a été développée sous forme d'une application web accessible à la racine du projet. Ce tableau de bord permet de visualiser en temps réel les informations essentielles du système.

- **Cartes de statut :** affichage de l'état de l'application (UP/DOWN), du temps de fonctionnement (uptime) ainsi que des performances (temps de réponse simulé).
- **Informations de build :** présentation de la version actuelle, du hash du commit Git ainsi que de la date du dernier déploiement.
- **Visualisation du pipeline :** représentation simplifiée des étapes du pipeline CI/CD (Code → Test → Docker → Déploiement).
- **Page des tests :** affichage des résultats des tests unitaires réalisés avec Jest.

12.2 Endpoints API de Monitoring

Afin d'améliorer l'observabilité du système, plusieurs endpoints REST ont été ajoutés :

- `GET /api/health` : vérifie l'état global de l'application.
- `GET /api/version` : retourne la version actuelle de l'application.
- `GET /api/metrics` : fournit des statistiques en temps réel telles que le nombre de requêtes traitées et l'uptime.
- `GET /api/build-info` : expose les métadonnées du build (commit, date, statut du pipeline).

Ces endpoints permettent une intégration future avec des outils de supervision tels que Prometheus ou Grafana.

12.3 Refactoring et Organisation du Code

Une restructuration complète du projet a été réalisée afin d'améliorer la maintenabilité et la lisibilité du code :

- `src/routes/` : définition des routes de l'application.
- `src/controllers/` : gestion de la logique métier.
- `src/services/` : implémentation des services (calculs, métriques).
- `src/public/` : ressources statiques pour l'interface utilisateur.

Cette architecture respecte les bonnes pratiques de séparation des responsabilités.

12.4 Optimisation Docker

Le processus de conteneurisation a été amélioré afin de rendre les images plus efficaces et informatives :

- Utilisation de **multi-stage build** pour réduire la taille de l'image finale.
- Injection de métadonnées (version, commit, date) via les instructions `ARG` et `ENV`.
- Amélioration du fichier `docker-compose.yml` pour garantir la cohérence entre build et exécution.

12.5 Amélioration du Pipeline CI/CD

Le pipeline GitHub Actions a été enrichi pour automatiser davantage le cycle de vie de l'application :

- Transmission automatique du hash du commit et de la date lors du build Docker.
- Validation stricte : le déploiement est conditionné par la réussite de l'ensemble des tests unitaires.
- Renforcement de la fiabilité du processus de livraison continue.

12.6 Documentation et Tests

La documentation du projet a été mise à jour afin de refléter les nouvelles fonctionnalités :

- Mise à jour du fichier `README.md` avec les nouveaux endpoints et fonctionnalités.
- Ajout de tests unitaires couvrant les nouvelles routes et services.
- Vérification du bon fonctionnement du dashboard via des tests automatisés.

Ces améliorations garantissent une meilleure qualité logicielle et facilitent la prise en main du projet.

12.7 Apport de ces améliorations

Ces évolutions permettent de transformer une simple application en une solution DevOps complète, offrant :

- Une meilleure visibilité sur l'état du système
- Une automatisation fiable des déploiements
- Une architecture maintenable et évolutive
- Une préparation à une mise en production réelle

Chapitre 13

Conclusion

Ce projet a permis de démontrer qu'une automatisation robuste est accessible même pour de petites applications. L'intégration de Docker et GitHub Actions réduit considérablement les risques d'erreur humaine lors du déploiement.

Chapitre 14

Perspectives d'Amélioration

Pour aller plus loin, nous pourrions :
— Intégrer un orchestrateur comme **Kubernetes**.
— Ajouter une analyse statique de code (SonarQube).
— Mettre en place une surveillance en temps réel avec **Prometheus/Grafana**.

Glossaire DevOps

CI (Continuous Integration) : Pratique de fusionner les modifications de code fréquemment dans un dépôt central.

CD (Continuous Deployment) : Automatisation du déploiement de chaque modification validée en production.

Conteneur : Unité standard de logiciel qui empaquette le code et ses dépendances.

Pipeline : Suite d'étapes automatisées pour livrer un logiciel.